

Extending MaxRL to Continuous Rewards with Entropy-Regularization

ECEN 743 Final Project Presentation

Sungi Hong Junyu Leng Qianhui Ye

Texas A&M University

Spring 2026

Presentation Roadmap

- 1 Motivation: sparse-reward decision problems and MaxRL
- 2 Problem: why binary MaxRL does not directly work for continuous rewards
- 3 Proposed method: continuous reward-mass normalization + entropy regularization
- 4 Experiments: LunarLander, GSM8K, Hopper and Walker
- 5 Key findings, limitations, and future directions

Main message

Continuous reward normalization gives MaxRL a usable learning signal before binary successes become common.

1. Sparse-Reward Sequential Decision Problems

Setting: the model receives delayed or terminal feedback after generating a trajectory.

Examples include:

- Mathematical reasoning: whether the final answer is correct
- Code generation: whether the solution passes unit tests
- Path planning/control: whether the agent reaches a goal or lands safely

Standard RL starts from reward maximization:

$$J_{\text{RL}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{\tau \sim \pi_{\theta}(\cdot|x)} [r(\tau, x)].$$

- x : task instance
- τ : generated trajectory
- $r(\tau, x)$: reward or correctness feedback

2. From Reward Maximization to Success Probability

In binary sparse-reward settings,

$$r(\tau, x) \in \{0, 1\},$$

where $r(\tau, x) = 1$ if trajectory τ solves instance x .

Define the success probability

$$p_\theta(x) := \Pr_{\tau \sim \pi_\theta(\cdot|x)} (r(\tau, x) = 1).$$

Since the reward is binary,

$$\mathbb{E}_{\tau \sim \pi_\theta(\cdot|x)} [r(\tau, x)] = p_\theta(x), \quad J_{\text{RL}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} [p_\theta(x)].$$

Interpretation

Standard policy gradient mainly optimizes [pass@1](#): the probability of success in one attempt.

3. What is MaxRL and Why Study It?

MaxRL idea: instead of maximizing success probability directly, optimize a likelihood-style objective:

$$J_{\text{ML}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}}[\log p_{\theta}(x)].$$

For a single task x , let $p := p_{\theta}(x)$. Then

$$\log p = - \sum_{k=1}^{\infty} \frac{(1-p)^k}{k} = - \sum_{k=1}^{\infty} \frac{\text{fail@ } k(x)}{k}.$$

Why this matters:

- $(1-p)^k$ is the probability that the model fails on all k tries.
- MaxRL emphasizes reducing repeated failures across multiple samples.
- This is natural for reasoning, code generation, and planning tasks where we can sample several candidate trajectories.

Tajwar et al. (2026), *Maximum Likelihood Reinforcement Learning*.

4. Our Gap: MaxRL is Binary, but Many Rewards are Continuous

Original MaxRL

- Designed for $r_i \in \{0, 1\}$.
- Normalizes by the number of successful samples:

$$\frac{1}{K} \sum_{i=1}^N r_i \nabla \log \pi_{\theta}(\tau_i), \quad K = \sum_{i=1}^N r_i.$$

Problem in control tasks

- Rewards are dense or continuous.
- Binary thresholding loses useful information.
- If $K = 0$, the update has **zero gradient**.

$K = 0 \Rightarrow$ no useful update.

Research question

Can we keep the likelihood-inspired normalization idea, but make it work for continuous rewards?

5. Standard Policy Gradient Baseline

For a trajectory $\tau_i = (s_{i,0}, a_{i,0}, \dots, s_{i,T})$, define the discounted return

$$R_i = \sum_{t=0}^T \gamma^t r_{i,t}.$$

The standard policy-gradient estimator is

$$\nabla J_{\text{RL}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N R_i \nabla \log \pi_{\theta}(\tau_i), \quad \log \pi_{\theta}(\tau_i) = \sum_t \log \pi_{\theta}(a_{i,t} | s_{i,t}).$$

Limitation

The normalization by N treats the batch size as the scale, even when only a few trajectories contain the strongest learning signal.

6. Proposed: Continuous-Reward MaxRL

Key idea: replace success-count normalization with **reward-mass normalization**.

When returns may be negative, first shift them to nonnegative values:

$$\tilde{R}_i = \max \left(R_i - \min_j R_j, 0 \right).$$

Then define the normalized trajectory weight

$$w_i = \frac{\tilde{R}_i}{\sum_j \tilde{R}_j + \epsilon}.$$

The resulting objective is a weighted log-likelihood loss:

$$L(\theta) = - \sum_{i=1}^N w_i \log \pi_{\theta}(\tau_i).$$

Effect

Higher-return trajectories receive larger updates, while the method remains defined even when no binary successes occur.

7. Entropy-Regularized Continuous MaxRL

Reward-normalized updates may over-concentrate on a few high-return trajectories.

We add policy entropy to encourage exploration:

$$J_\lambda(\theta) = J_{\text{CR-MaxRL}}(\theta) + \lambda H(\pi_\theta),$$

where

$$H(\pi_\theta) = -\mathbb{E}_{\tau \sim \pi_\theta} [\log \pi_\theta(\tau)].$$

The implemented loss is

$$L(\theta) = -\sum_i w_i \log \pi_\theta(\tau_i) - \lambda H(\pi_\theta).$$

Important empirical point

Entropy helps, but the coefficient must be tuned. Too much entropy can dominate the reward-driven update.

8. Algorithm Summary

- 1 Sample a batch of trajectories $\{\tau_i\}_{i=1}^N$ from the current policy π_θ .
- 2 Compute returns $R_i = \sum_t \gamma^t r_{i,t}$.
- 3 Transform returns to nonnegative reward masses $\tilde{R}_i$.
- 4 Compute normalized weights

$$w_i = \frac{\tilde{R}_i}{\sum_j \tilde{R}_j + \epsilon}.$$

- 5 Update the policy using

$$L(\theta) = - \sum_i w_i \log \pi_\theta(\tau_i) - \lambda H(\pi_\theta).$$

Comparison

Method	Weighting idea	Main weakness
REINFORCE	Raw return / batch size	High variance
Binary MaxRL	Successful samples only	Zero update when no success
Continuous MaxRL	Reward mass	Needs exploration control

9. Experimental Setup: LunarLander-v3

Environment

- Gymnasium LunarLander-v3
- Dense intermediate rewards with sparse success signal
- Success defined using a return threshold
- Useful for testing whether Continuous MaxRL can learn before successful trajectories are frequently observed

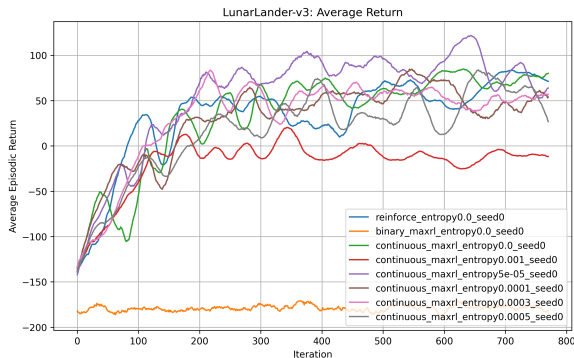
Compared Methods

- REINFORCE
- Binary MaxRL
- Continuous MaxRL
- Entropy-regularized Continuous MaxRL

Implementation

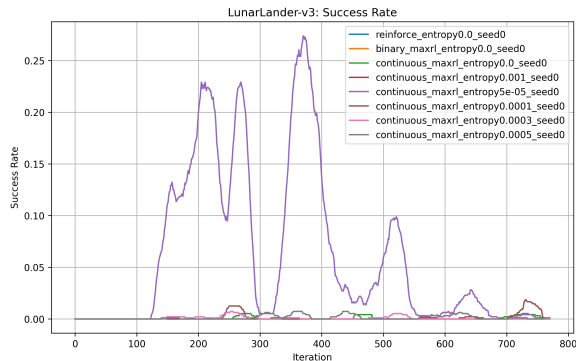
- Policy: 2-layer MLP
- Activation: Tanh
- Hidden size: 128
- Optimizer: Adam
- Learning rate: 10^{-3}
- 8 parallel environments
- Discount factor: $\gamma = 0.99$
- Maximum episode length: 500
- Training iterations: 800

10. LunarLander Result: Average Return



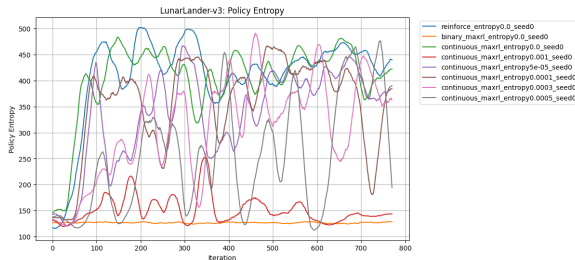
- Continuous MaxRL achieves higher and smoother average return than REINFORCE.
- Binary MaxRL performs poorly because successful trajectories are rare early in training.
- Reward normalization emphasizes relatively good trajectories without requiring binary success.

11. LunarLander Result: Success Rate



- Entropy-regularized Continuous MaxRL with a small coefficient shows the clearest non-trivial success spikes.
- Return improvement does not always immediately imply high success rate under a strict threshold.
- The method learns useful partial behaviors before consistently crossing the success threshold.

12. LunarLander Result: Policy Entropy



- REINFORCE often maintains high entropy but has unstable returns.
- Continuous MaxRL balances reward-seeking and exploration.
- Too much or too little entropy regularization can hurt learning stability.

13. Entropy Coefficient Sensitivity

Sweep:

$$\lambda \in \{5 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}, 5 \times 10^{-4}, 10^{-3}\}.$$

- Small entropy regularization can improve exploration and success discovery.
- The best LunarLander setting in the report was around $\lambda = 5 \times 10^{-5}$.
- Larger coefficients can degrade return and success rate by overwhelming the reward-normalized gradient.

Theoretical intuition

$$\nabla J_{\lambda_1}(\theta) - \nabla J_{\lambda_2}(\theta) = (\lambda_1 - \lambda_2) \nabla H(\pi_\theta).$$

If λ is too large, the entropy term can dominate the reward-driven direction.

14. MuJoCo Locomotion Environments

Why these environments? Hopper-v5 and Walker2d-v5 are standard MuJoCo continuous-control benchmarks. Both tasks require the agent to learn stable forward locomotion from high-dimensional continuous states and continuous torque actions.

Hopper-v5

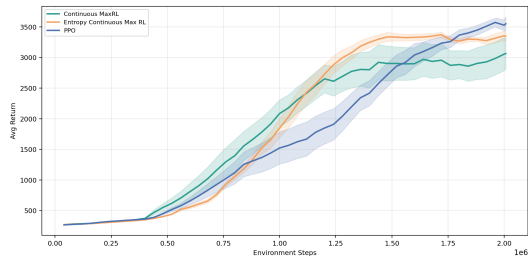
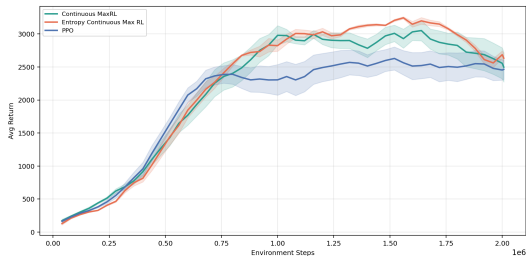
- A 2D one-legged robot that must hop forward while maintaining balance.
- Observation space: 11-dimensional continuous state.
- Action space: 3-dimensional continuous torque control.
- The task is sensitive to unstable updates: the agent can easily fall, leading to early termination.
- Useful for testing whether MaxRL can emphasize high-return trajectories and improve robustness.

Evaluation: We compare PPO, Continuous MaxRL, and Entropy Continuous MaxRL under the same training budget of 2M environment steps.

Walker2d-v5

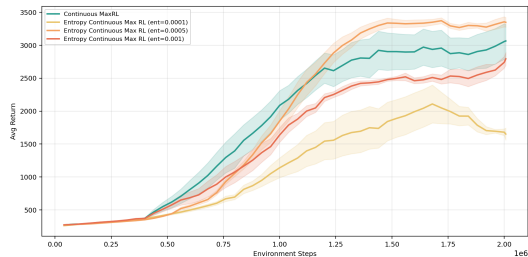
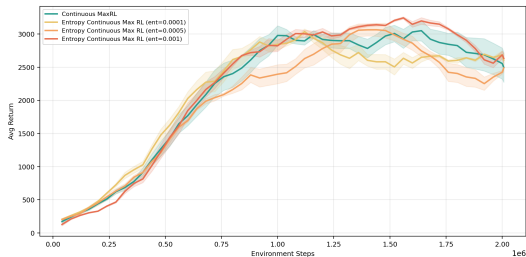
- A 2D bipedal robot that must learn coordinated walking.
- Observation space: 17-dimensional continuous state.
- Action space: 6-dimensional continuous torque control.
- More complex than Hopper because it requires coordination across two legs.
- Useful for evaluating whether MaxRL remains competitive in more complex locomotion tasks.

15. Hopper-v5 and Walker2d-v5 Results



- On Hopper-v5, Entropy Continuous MaxRL with $\lambda = 0.0005$ achieves the best result: 2889.49 ± 391.30 , outperforming PPO (2452.39 ± 900.18) and Continuous MaxRL (2340.40 ± 914.41).
- On Walker2d-v5, PPO has the highest mean return (3330.53 ± 1091.93), but Entropy Continuous MaxRL is very close (3276.50 ± 108.26) and much more stable.
- Hopper and Walker show complementary conclusions: entropy regularization clearly improves both return and robustness on Hopper, while on Walker it mainly improves robustness and keeps performance competitive with PPO.

16. Effect of the Entropy Coefficient



- We compare Continuous MaxRL with entropy coefficients $\lambda \in \{0.0001, 0.0005, 0.001\}$.
- In both Hopper-v5 and Walker2d-v5, the intermediate setting $\lambda = 0.0005$ gives the strongest overall result.
- On Hopper, $\lambda = 0.0005$ improves return from 2340.40 ± 914.41 to 2889.49 ± 391.30 ; on Walker, it improves stability substantially while slightly increasing mean return.
- Too small or too large entropy coefficients lead to weaker performance or larger variance, showing that entropy regularization is beneficial but strongly parameter-sensitive.

17. Additional Test: GSM8K Reasoning with distilgpt2

Goal: test whether MaxRL-style weighting can be applied beyond control tasks.

Setup

- Dataset: GSM8K math reasoning
- Base model: distilgpt2
- Batch size: 8 prompts
- 8 samples per prompt
- 200 training iterations
- AdamW, learning rate 10^{-5}

Reward design

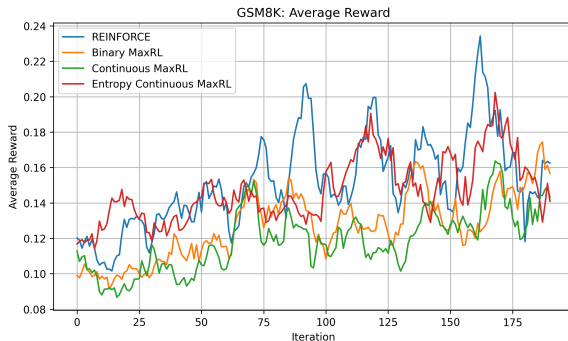
$$r_i = \begin{cases} 0.05, & \text{if numerical answer is produced,} \\ 0, & \text{otherwise,} \end{cases}$$

plus 1.0 for exact numerical match.

Reason for partial reward

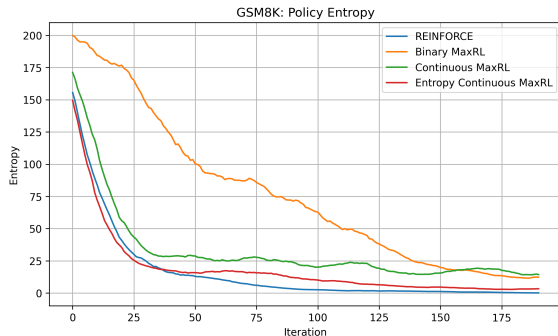
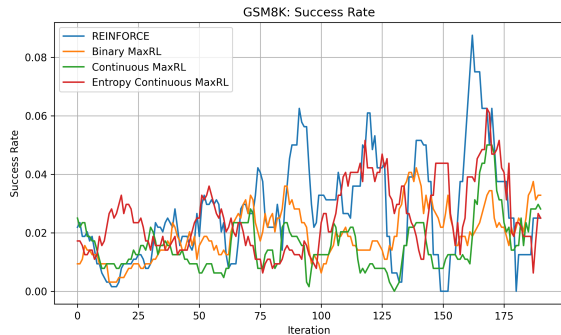
Exact correctness is extremely sparse for a small base model, so partial reward gives a denser signal.

18. GSM8K Result: Average Reward



- REINFORCE reaches high reward spikes but is unstable.
- Entropy Continuous MaxRL improves over non-regularized Continuous MaxRL.
- The task remains difficult because distilgpt2 has weak mathematical reasoning ability.

19. GSM8K Result: Success Rate and Entropy



- REINFORCE achieves the highest success spike, but the improvement is not sustained.
- Entropy Continuous MaxRL reaches a higher maximum success rate than Binary and non-entropy Continuous MaxRL.
- Entropy collapse suggests premature concentration on a narrow set of generations.

20. Key Findings

Finding 1: Binary MaxRL can fail under rare success

When no successful trajectory appears in a batch, the binary MaxRL update vanishes.

Finding 2: Continuous normalization gives denser feedback

Reward-mass weighting uses relative trajectory quality, so learning can begin before strict successes appear.

Finding 3: Entropy regularization is useful but sensitive

A small entropy term improves exploration; an overly large one disrupts reward-driven improvement.

Finding 4: Results are promising but task-dependent

Continuous MaxRL works clearly in LunarLander, but GSM8K requires stronger models or better reward/verifier design.

21. Limitations and Future Work

Limitations

- Experiments use limited seeds and relatively small training budgets.
- GSM8K uses distilgpt2, which is not a strong reasoning model.
- Reward shaping for language tasks is still simple.
- Entropy coefficient tuning is sensitive.

Future directions

- Test with stronger language models and verifier-based rewards.
- Combine Continuous MaxRL with PPO or other stabilized RL algorithms.
- Develop adaptive entropy schedules.
- Analyze bias/variance properties of reward-mass normalized estimators.

Team Contributions

Sungi Hong

- Developed the entropy-regularized MaxRL mathematical formulations
- Developed the main implementation code
- Conducted LunarLander-v3 and distilgpt2 experiments
- Wrote the problem formulation and appendix theoretical insight

Junyu Leng

- Conducted Hopper-v5 and Walker2d-v5 experiments
- Analyzed the performance of entropy-regularized Continuous MaxRL with PPO
- Developed appendix theoretical insight

Qianhui Ye

- Conducted Qwen-based language-model experiments
- Conducted Acrobot-v1 entropy experiments

All members

- Participated in the literature review
- Developed the Continuous MaxRL
- Reviewed and revised the full project report, presentation, and code

Continuous-reward MaxRL extends likelihood-inspired RL beyond binary success/failure feedback.

- We replace success-count normalization with reward-mass normalization.
- We add entropy regularization to prevent premature collapse.
- Experiments show improved stability and performance in continuous-control settings.
- Reasoning experiments suggest the same idea is possible, but require stronger reward signals and models.

Thank you!